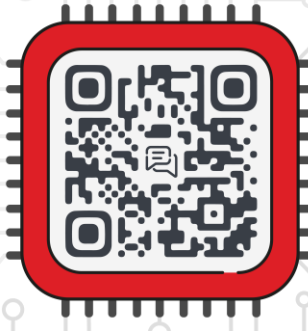




8



21



3150 sp

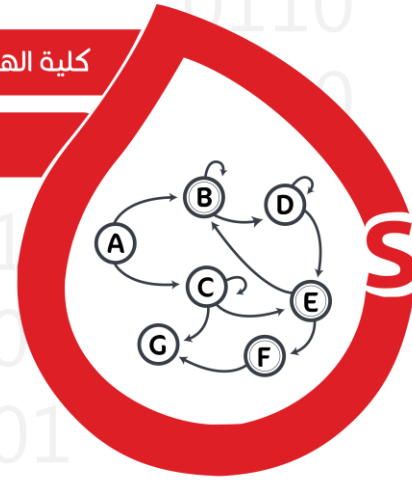
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

## Reducing Grammar & Turing Machine

د. باسم قصيبة



RB Informatics; 24/5/2026

محتوى مجاني غير مخصص للبيع التجاري

اللغات الصورية

بدايةً، سنراجع معاً بعض الأفكار من المحاضرة السابقة، فهذه الأفكار أساسية جداً لفهم آلية عمل المترجمات (Compilers).

### 1. خوارزمية CYK وطرق الاشتقاق (Parsing)

لفهم كيف يقوم المترجم بإعراب الجمل البرمجية، علينا أن نميز بين طريقتين أساسيتين للاشتقاق:

1. الاشتقاق من الأعلى للأسفل (Top-Down):

تخيل أنك تبدأ من القمة، أي من "رمز البداية" (Start Symbol)، ثم تطبق قواعد اللغة نزولاً خطوة بخطوة في محاولة لتوليد الكلمة المطلوبة. وإذا واجهت بدائل للقاعدة، تقوم بتجربة البديل المناسب للوصول إلى الكلمة.

2. الاشتقاق من الأسفل للأعلى (Bottom-Up):

هنا العملية عكسية تماماً! أنت تبدأ من الكلمة نفسها الموجودة لديك (مجموعة المفردات أو Tokens)، وتحاول تجميعها وتركيبها صعوداً لتصل إلى "رمز البداية"، إذا نجحت بالوصول إليه، فالكلمة صحيحة ومقبولة.

ملاحظة هامة: خوارزمية CYK التي درسناها تعتمد على أسلوب Bottom-Up.

■ تذكرة بقواعد تشومسكي (CNF):

لنتذكر معاً شرطاً أساسياً لعمل خوارزمية CYK، وهو أن تكون القواعد مصاغة وفق "الشكل النظامي لتشومسكي" (Chomsky Normal Form) وهذا يعني أن كل قاعدة يجب أن تنتج إما:

1. رمزاً طرفياً واحداً (Terminal).

2. أو رمزين غير طرفيين فقط (Non-terminals 2).

**السبب:** هذا الشكل الثنائي يسمح للخوارزمية بتركيب المفردات "مثنى مثنى" بشكل تدريجي ومرحلي حتى نصل إلى رمز البداية.

## 2. اللغة الغامضة (Ambiguous Grammar)

ما هي اللغة الغامضة؟

تخيل أنك تقرأ جملة تحتل معنيين؛ هذا بالضبط ما يعنيه الغموض هنا. اللغة تعتبر "غامضة" عندما تسمح لك قواعدها باشتقاق نفس الجملة بأكثر من طريقة صحيحة (أي الوصول لنفس النتيجة بطريقتين مختلفتين).

مثال عملي لتوضيح الفكرة:

لدينا القواعد الرياضية التالية:

$Expression \rightarrow Expression + E \mid Expression * E \mid Digits$

(حيث تمثل Digits الأرقام من 0 إلى 9).

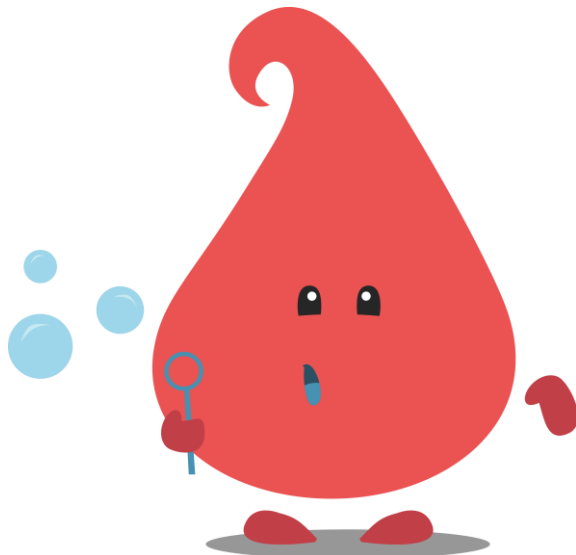
عندما نطلب من المترجم إعراب الجملة:  $9 * 7 + 5$ ، سنواجه مشكلة الغموض؛ فالقواعد تتيح للمترجم خيارين:

1. أن يجمع  $(7+5)$  أولاً ثم يضرب الناتج بـ 9.
2. أو أن يضرب  $(9*7)$  أولاً ثم يجمع الناتج مع 5.

مشكلة اللغات الغامضة في المترجمات:

المشكلة هنا أن المترجم يحتاج إلى "الحتمية" (Determinism)، يجب أن يكون أمامه في كل خطوة حالة واحدة وخيار واحد فقط لينفذه دون تردد.

في حال وجود غموض، لن يعرف المترجم أولوية العمليات الحسابية وسيؤدي ذلك إلى أخطاء فادحة. لذلك، يجب علينا صياغة القواعد بذكاء لتعكس الأولويات الصحيحة (كأن نُجبر اللغة على تنفيذ الضرب قبل الجمع عبر وضع قواعد الضرب في أسفل شجرة الاشتقاق لتدخل المكس أولاً)، وبذلك نلغي أي خيارات محيرة أمام المترجم.



### 3. العودية اليسارية (Left Recursion)

ما هي العودية اليسارية؟

تحدث هذه الحالة عندما تبدأ القاعدة باستدعاء نفسها مباشرة في أقصى يسار الجزء المشتق.  
مثال:

$$A \rightarrow A C$$

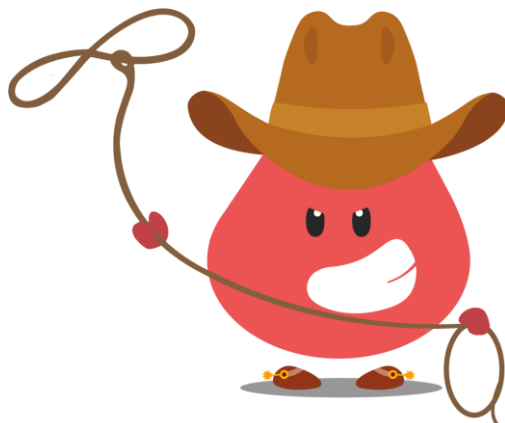
المشكلة التي تسببها:

من المشاكل الكبيرة التي ستواجهنا عند بناء المترجمات بطريقة (Top-Down)، أن المترجم يُحول هذه القواعد إلى دوال برمجية (Functions) تعتمد على المكس. إذا تركت العودية اليسارية، ستقوم الدالة  $A$  باستدعاء نفسها بشكل لا نهائي دون أن تستهلك أو تقرأ أي رمز من سلسلة المُدخلات (Input) ببساطة، سيدخل البرنامج في حلقة مفرغة تنتهي بامتلاء المكس وانهايار المترجم (Stack Overflow).

### 4. كيف تُطبق هذه القواعد فعلياً؟

لنتذكر كيف يفهم الحاسوب كل هذا باستخدام "المكس" (Stack) وآلة الأوتومات (Pushdown Automata):

- نقوم بتحويل قواعد اللغة (Context-Free Grammar) ووضعها داخل المكس.
- كلما ظهر "رمز غير طرفي" (Non-terminal) في قمة المكس، نستبدله بالبديل المناسب بناءً على ما نقرأه من شريط المُدخلات (Input).
- بمجرد أن يظهر "رمز طرفي" (Terminal) في قمة المكس مطابق للرمز الموجود في المُدخل، نقوم بعملية مطابقة (Matching) ونحذفهما معاً.
- تستمر هذه العملية حتى يفرغ المكس تماماً، وهي الفكرة الأساسية التي يدور حولها بناء المترجمات.



لماذا نزيل العودية اليسارية؟

عند بناء المترجمات التي تعتمد على أسلوب الإعراب من الأعلى للأسفل (Top-Down)، يقوم المترجم بتحويل كل قاعدة إلى دالة برمجية (Function) إذا كانت القاعدة تستدعي نفسها في البداية مثل  $A \rightarrow AC$ ، فإن الدالة ستتدعي نفسها بشكل متكرر ولا نهائي دون أن تقرأ أي رمز من شريط المُدخلات، مما يؤدي في النهاية إلى طفح المكس وانهيال المترجم (Stack Overflow)، لذا، يجب إعادة صياغة القواعد للتخلص من هذه المشكلة.

#### القاعدة العامة لإزالة العودية اليسارية:

لحل هذه المشكلة، نقوم بفصل أجزاء القاعدة إلى قسمين:

- قسم يسبب العودية نسميه  $(\alpha)$
- وقسم لا يسبب العودية نسميه  $(\beta)$

إذا كان لدينا قاعدة بالشكل التالي:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2 \mid \beta_3$$

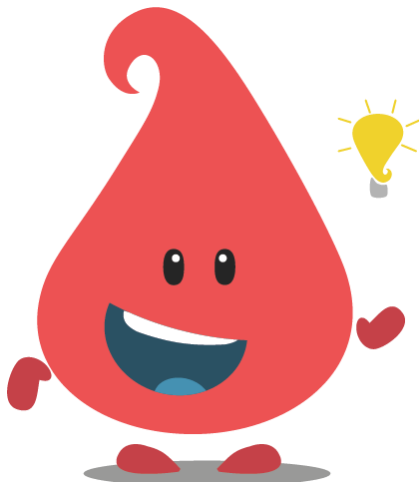
فإننا نزيل العودية اليسارية بإضافة متحول جديد كلياً سنرمز له بـ  $(X)$ ، وتصبح القواعد الجديدة كالتالي:

1. القاعدة الأصلية:  $(A)$  تأخذ جميع أجزاء  $\beta$  متبوعة بالمتحول الجديد  $X$ .

$$A \rightarrow \beta_1 X \mid \beta_2 X \mid \beta_3 X$$

2. المتحول الجديد:  $(X)$  يأخذ جميع أجزاء  $\alpha$  متبوعة بـ  $X$ ، بالإضافة إلى حالة التوقف  $\epsilon$ .

$$X \rightarrow \alpha_1 X \mid \alpha_2 X \mid \epsilon$$



ليكن لدينا المثال التالي لقاعدة رياضية تحتوي على عودية يسارية:

$$E \rightarrow E + E \mid E * E \mid \text{digits}$$

لتطبيق قانون الإزالة على هذا المثال، سنتبع الخطوات التالية:

### 1. الخطوة الأولى: تحديد أجزاء الـ $\alpha$ والـ $\beta$

نبحث عن البدائل التي تبدأ بنفس رمز القاعدة ( $E$ ) لنستخرج منها  $\alpha$ ، والبدائل التي لا تبدأ بـ ( $E$ ) لنستخرج منها  $\beta$ :

#### الأجزاء المسببة للعودية: ( $\alpha$ )

- الرمز الموجود في أقصى اليسار هو ( $E$ ) وهو الرمز المسبب للعودية
- إذا قمنا باستبعاد هذا الـ  $E$  الأول، ماذا يتبقى من الجملة؟ المتبقي هو  $E +$  فقط.
- لذلك نعتبر أن  $\alpha_1 = +E$
- نستبعد الرمز  $E$  الموجود في أقصى اليسار.
- المتبقي من الجملة هو  $E *$  فقط.
- لذلك نعتبر أن  $\alpha_2 = *E$

#### الأجزاء السليمة: ( $\beta$ )

$$\beta = \text{digits}$$

### 2. الخطوة الثانية: صياغة قاعدة $E$ الجديدة

حسب القانون، نأخذ الجزء السليم  $\beta$  ونثبته بالمتحول الجديد  $X$ . تصبح القاعدة:

$$E \rightarrow \text{digits}X$$

### 3. الخطوة الثالثة: صياغة قاعدة المتحول الجديد $X$

حسب القانون، نأخذ أجزاء  $\alpha$  التي استخرجناها، ونضع بجانب كل منها المتحول  $X$ ، ثم نضيف ايسيلون ( $\epsilon$ ) في النهاية كحالة توقف. تصبح القاعدة:

$$X \rightarrow + E X \mid * E X \mid \epsilon$$

النتيجة النهائية

القاعدة الغامضة والمسببة للعودية:

$$E \rightarrow E + E \mid E * E \mid \text{digits}$$

أصبحت بعد معالجتها قواعد حتمية نظيفة لا تسبب انهماكاً للمترجم:

$$E \rightarrow \text{digits}X$$

$$X \rightarrow + E X \mid * E X \mid \epsilon$$

## 1 - حذف الرموز عديمة الفائدة Useless Grammar Symbol

■ ما هي الرموز عديمة الفائدة (Useless Symbols) ؟

الرمز عديم الفائدة هو أي متحول (Variable / Non-terminal) لا يمكن من خلاله اشتقاق أي سلسلة من الرموز الطرفية (Terminals) سواء بشكل مباشر أو غير مباشر.

■ لماذا يجب إزالتها؟

- لأن هذه المتحولات لا تساهم أبداً في بناء أو إنتاج أي جملة صحيحة في اللغة.
- بقاؤها يؤدي إلى تضخيم القواعد ودخول المترجم في حلقات مغلقة أو حالات بئر (Sink States) تعطل عمل المترجم.

خوارزمية تحديد وإزالة الرموز عديمة الفائدة ، تتم العملية على مرحلتين أساسيتين:

1. إيجاد مجموعة الرموز المفيدة  $(V_i)$  :

- المجموعة الأولى  $(V_1)$ : نجمع فيها كل المتحولات التي تنتج رمزا طرفياً (Terminal) بشكل مباشر، أو تنتج  $\epsilon$  لأن  $\epsilon$  تعامل معاملة الرمز الطرفي.
- المجموعات التالية  $(V_2, V_3, \dots)$ : نضيف تكرارياً المتحولات التي تعتمد في اشتقاقها على رموز موجودة في المجموعة السابقة بالإضافة للرموز الطرفية.
- نتوقف عندما تتساوى مجموعتان متتاليتان  $(V_{k+1} = V_k)$ .

2. حذف الرموز عديمة الفائدة وقواعدها:

1. الرموز التي لم تظهر في المجموعة المفيدة النهائية تعتبر رموزاً عديمة الفائدة.
2. نلغي قواعد هذه الرموز بالكامل، ثم نحذف أي بديل في القواعد الأخرى يظهر فيه أحد هذه الرموز عديمة الفائدة.



It's LeviOsa, not LeviosAh!

ليكن لدينا القواعد الآتية:

$$\begin{aligned} A &\rightarrow AT \mid T \mid U \\ T &\rightarrow bT \mid aCX \mid \varepsilon \\ U &\rightarrow bU \mid abW \mid \varepsilon \\ B &\rightarrow abU \mid bW \mid a \\ C &\rightarrow bT \mid UC \\ W &\rightarrow bUX \mid TUC \\ X &\rightarrow aXU \mid bTYZ \\ Y &\rightarrow aYb \mid aW \\ Z &\rightarrow XU \mid aZY \end{aligned}$$

### 1. الخطوة الأولى: تحديد الرموز المفيدة تكرارياً

■ إيجاد المجموعة الأولى ( $V_1$ ) :

نبحث عن المتحولات التي تنتج رموزاً طرفية أو  $\varepsilon$  بشكل مباشر:

$$\begin{aligned} T &\rightarrow \varepsilon \\ U &\rightarrow \varepsilon \\ B &\rightarrow a \end{aligned}$$

$$V_1 = \{T, U, B\}$$

■ إيجاد المجموعة الثانية ( $V_2$ ) :

نبحث عن المتحولات التي تشتق رموزاً باستخدام عناصر  $V_1$ :

$$\circ A \rightarrow T \mid U \quad (\text{لأن } T \text{ و } U \text{ ينتميان إلى } V_1) \leq \text{نضيف } A$$

$$\circ C \rightarrow bT \quad (\text{لأن } T \text{ ينتمي إلى } V_1) \leq \text{نضيف } C$$

$$V_2 = \{T, U, B, A, C\}$$

■ إيجاد المجموعة الثالثة ( $V_3$ ) :

نبحث عن المتحولات التي يمكنها الآن إنتاج سلاسل طرفية بوجود عناصر  $V_2$ :

$$\circ W \rightarrow TUC \quad (\text{حيث } T \text{ و } C \text{ و } U \text{ ينتميان إلى } V_2) \leq \text{نضيف } W$$

$$V_3 = \{T, U, B, A, C, W\}$$

■ إيجاد المجموعة الرابعة ( $V_4$ ) :

$$\circ Y \rightarrow aW \quad (\text{لأن } W \text{ ينتمي إلى } V_3) \leq \text{نضيف } Y$$

$$V_4 = \{T, U, B, A, C, W, Y\}$$

■ إيجاد المجموعة الخامسة: ( $V_5$ )  
عند فحص باقي المتحولات ( $X$  و  $Z$ ):

$$X \rightarrow a X U \mid b T U Z \quad \circ$$

$$Z \rightarrow X U \mid a Z Y \quad \circ$$

لا يمكن لأي منهما الوصول إلى رموز طرفية مطلقاً.

$$V_5 = \{T, U, B, A, C, W, Y\} = V_4$$

الرموز المفيدة  $\{A, T, U, B, C, W, Y\}$

الرموز عديمة الفائدة  $\{X, Z\}$

2. الخطوة الثانية: حذف الرموز عديمة الفائدة وقواعدها

1. حذف القواعد الخاصة بـ  $X$  و  $Z$  بالكامل:

$$X \rightarrow a X U \mid b T U Z \quad \circ$$

$$Z \rightarrow X U \mid a Z Y \quad \circ$$

2. حذف البدائل التي يظهر فيها  $X$  أو  $Z$  في بقية القواعد:

$$\circ \text{ في القاعدة } T \text{ نحذف البديل } a C X \text{ لتصبح:}$$

$$T \rightarrow b T \mid \varepsilon$$

$$\circ \text{ في القاعدة } W \text{ نحذف البديل } b U X \text{ لتصبح:}$$

$$W \rightarrow T U C$$

القواعد النهائية بعد التنظيف والتخلص من الرموز عديمة الفائدة :

$$A \rightarrow A T \mid T \mid U$$

$$T \rightarrow b T \mid \varepsilon$$

$$U \rightarrow b U \mid a b W \mid \varepsilon$$

$$B \rightarrow a b U \mid b W \mid a$$

$$C \rightarrow b T \mid U C$$

$$W \rightarrow T U C$$

$$Y \rightarrow a Y b \mid a W$$

Hasta el final!



## 2- حذف الرموز غير القابلة للوصول Inaccessible Symbols

الرموز غير القابلة للوصول هي تلك المتحولات (Variables) التي لا يمكن الوصول إليها أبداً أو المرور بها في حال بدأنا عملية الاشتقاق من "رمز البداية (Start Symbol)"  
بمعنى آخر، هي قواعد قد تكون صحيحة ومفيدة في ذاتها، لكن المترجم لن يستخدمها إطلاقاً لأن مسار البرنامج الأساسي لا يمر بها ولا يستدعيها أبداً، وبالتالي تعتبر "كوداً ميتاً (Dead Code)" يجب حذفه لتنظيف القواعد وتسريع عمل المترجم.

### خطوات الخوارزمية:

تعتمد هذه الخوارزمية على تتبع المسار انطلاقاً من نقطة البداية، وإضافة أي رمز نمر عليه إلى مجموعة "الرموز التي يمكن الوصول إليها (Accessible Symbols)"، وذلك وفق الخطوات التالية:

1. نُنشئ المجموعة الأولى  $V_1$  ونضع بداخلها "رمز البداية" فقط.
2. ننظر إلى القواعد الخاصة بالرموز الموجودة في المجموعة الحالية، وأي متحول جديد يظهر على يمين السهم، نقوم بإضافته إلى المجموعة التالية  $V_2$ .
3. نكرر الخطوة السابقة (البحث داخل قواعد الرموز الجديدة المكتشفة) لتشكيل المجموعات  $V_3, V_4, \dots$  وهكذا.
4. نتوقف عندما نصل إلى مجموعة لا تضيف أي رمز جديد (أي عندما تتساوى مجموعتان متتاليتان).
5. في النهاية، أي رمز غير موجود في المجموعة الأخيرة يُعتبر "غير قابل للوصول" ونقوم بحذفه مع كامل قواعده.

### ملاحظة :

إذا لم يُصرح لك في نص المسألة أو الامتحان عن "رمز البداية (Start Symbol)"، فإنك دائماً تعتبر الرمز الموجود في أول قاعدة هو رمز البداية افتراضياً.

### سنتابع من النتيجة التي وصلنا لها في التمرين الأخير

في هذا التمرين، استند إلى القواعد التي نتجت معنا بعد تنظيفها من "الرموز عديمة الفائدة (Useless Symbols)" في الخطوة السابقة، القواعد التي لدينا هي:

$$\begin{aligned} A &\rightarrow AT \mid T \mid U \\ T &\rightarrow bT \mid \varepsilon \\ U &\rightarrow bU \mid abW \mid \varepsilon \\ B &\rightarrow abU \mid bW \mid a \\ C &\rightarrow bT \mid UC \\ W &\rightarrow TUC \\ Y &\rightarrow aYb \mid aW \end{aligned}$$

الهدف : إيجاد الرموز القابلة للوصول وحذف الباقي.

### خطوات الحل:

الخطوة الأولى المجموعة  $(V_1)$ :

بما أنه لم يُذكر رمز البداية، سنأخذ الرمز الأول في القواعد وهو  $A$ .

$$V_1 = \{A\}$$

الخطوة الثانية المجموعة  $(V_2)$ :

ننظر إلى قاعدة الرمز  $A$  لنتحقق من المتحولات التي يستدعيها:

القاعدة هي

$$A \rightarrow A T \mid T \mid U$$

نلاحظ ظهور المتحولين  $T$  و  $U$ . نقوم بإضافتهما.

$$V_2 = \{A, T, U\}$$

الخطوة الثالثة المجموعة  $(V_3)$ :

الآن نبحث داخل قواعد الرموز الجديدة  $T$  و  $U$  لنرى إلى أين ستأخذنا:

○ القاعدة  $T \rightarrow b T \mid \varepsilon$  لم يظهر أي متحول جديد

○ القاعدة  $U \rightarrow b U \mid a b W \mid \varepsilon$  ظهر المتحول  $W$  نقوم بإضافته.

$$V_3 = \{A, T, U, W\}$$

الخطوة الرابعة المجموعة  $(V_4)$ :

نبحث الآن داخل قاعدة الرمز الجديد الذي اكتشفناه:  $(W)$

○ القاعدة  $W \rightarrow T U C$  المتحولات  $T$  و  $U$  لدينا مسبقاً، لكن  $C$  متحول جديد نقوم بإضافته.

$$V_4 = \{A, T, U, W, C\}$$

الخطوة الخامسة (نهاية الخوارزمية):

نبحث داخل قاعدة الرمز الجديد:  $(C)$

○ القاعدة  $C \rightarrow b T \mid U C$  الرمز  $T$  و  $U$  موجودان لدينا بالفعل، ولم يظهر أي رمز جديد

هنا نتوقف الخوارزمية لأن المجموعة استقرت.

$$V_5 = \{A, T, U, W, C\}$$

النتيجة النهائية للتمرين:

المتحولات التي يمكن الوصول إليها هي  $\{A, T, U, W, C\}$

المتحولات التي لا يمكن الوصول إليها يجب حذفها مع قواعدها بالكامل هي  $\{B, Y\}$

ملاحظة:

في قواعد بناء المترجمات واختصار القواعد، الترتيب إجباري ومهم جداً. يجب عليك دائماً البدء بتمرين حذف الرموز عديمة الفائدة (Useless Variables)، وبعد الانتهاء منه تماماً، تقوم بتطبيق تمرين حذف الرموز غير القابلة للوصول (Inaccessible Variables) على القواعد الناتجة. لا يمكنك عكس هذا الترتيب إطلاقاً.

### 3- حذف قواعد الإبسيلون Epsilon Productions Removal

في اللغات الخالية من السياق (Context-Free Grammar)، تُعرف "قواعد الإبسيلون" بأنها القواعد التي تولد سلسلة فارغة  $\epsilon$  والمتحول الذي يستطيع توليد  $\epsilon$  (سواء بشكل مباشر أو غير مباشر) يُسمى المتحول القابل للتفريغ (Nullable Variable).

لماذا نحذف قواعد الإبسيلون؟

وجود  $\epsilon$  يعقد عمل خوارزميات الإعراب مثل (CYK)، لأن المترجم قد يدخل في مسارات اشتقاق وهمية لا تستهلك أي رموز من الدخل. لذلك، نقوم بحذفها لتصبح القواعد أكثر وضوحاً وحتمية.

القاعدة الذهبية للحذف:

*التعويض وليس الحذف المباشر!* نحن لا نمسح  $\epsilon$  ببساطة، بل نبحث عن كل قاعدة يظهر فيها "متحول قابل للتفريغ"، ونقوم بتوليد بدائل جديدة تغطي حالة غياب هذا المتحول (أي نعوض أثره وكأنه أصبح  $\epsilon$ )

■ خطوات الخوارزمية:

تتم العملية على مرحلتين أساسيتين:

1. اكتشاف المتحولات القابلة للتفريغ: (Nullable Variables)

- المجموعة الأولى ( $V_1$ ): نضع فيها المتحولات التي تولد  $\epsilon$  بشكل مباشر وصريح.
- المجموعات التالية ( $V_2, V_3, \dots$ ): نضيف إليها أي متحول يتكون الطرف الأيمن من قاعدته حصراً من متحولات ثبت أنها قابلة للتفريغ في المجموعات السابقة. نتوقف عند استقرار المجموعة.

2. تعويض الأثر وتوليد البدائل:

- نمر على جميع قواعد اللغة.
- في كل بديل (على يمين السهم) يحتوي على متحول قابل للتفريغ، نقوم بإنشاء "نسخة جديدة" من هذا البديل مع حذف هذا المتحول منه.
- إذا كان هناك أكثر من متحول قابل للتفريغ في نفس البديل، نأخذ جميع التشكيلات والاحتمالات الممكنة (حذف الأول، حذف الثاني، حذف كليهما).
- أخيراً، نحذف قاعدة الـ  $\epsilon$  الصريحة تماماً من اللغة.



طرح الدكتور مثلاً صغيراً قبل البدء بالتمرين الأساسي لتوضيح فكرة "التعويض":

$$S \rightarrow A A$$

$$A \rightarrow b \mid \varepsilon$$

### طريقة الحل:

نلاحظ بوضوح أن  $A$  يولد  $\varepsilon$  بشكل مباشر. (Nullable) الآن نذهب للقاعدة  $S$  التي تحتوي على  $A A$  ونأخذ جميع الاحتمالات:

- إذا جاءت  $A$  الأولى  $\varepsilon$ : يتبقى لدينا  $A$ .
  - إذا جاءت  $A$  الثانية  $\varepsilon$ : يتبقى لدينا  $A$ .
  - وعليه نضيف البديل  $A$  لقاعدة  $S$
- النتيجة بعد حذف الإبسيلون وتعويض الأثر:

$$S \rightarrow A A \mid A$$

$$A \rightarrow b$$

■ نتابع من التمرين السابق:

القواعد التي نتجت معنا بعد التنظيم من التمارين السابقة:

$$A \rightarrow A T \mid T U$$

$$T \rightarrow b T \mid \varepsilon$$

$$U \rightarrow b U \mid a b W \mid \varepsilon$$

$$C \rightarrow b T \mid U C$$

$$W \rightarrow T U C$$

ملاحظة: الرمز  $b$  هنا يمثل الرمز الطرفي (Terminal)

الخطوة 1: تحديد المتحولات القابلة للتفريغ (Nullable)

■ المجموعة الأولى ( $V_1$ ): نبحث عن يولد  $\varepsilon$  مباشرة:  
نجد أن  $T$  و  $U$  يقومان بذلك.

$$V_1 = \{T, U\}$$

■ المجموعة الثانية ( $V_2$ ): نبحث عن يولد مزيجاً من  $T$  و  $U$  فقط:

قاعدة  $A$  تحتوي على بديلين منفصلين ( $T$  و  $U$ ) بما أن  $T$  بحد ذاتها من عناصر  $V_1$  (أي nullable)، فهذا كافٍ

وحده لإثبات أن  $A$  أيضاً — nullable عبر المسار  $A \rightarrow T \rightarrow \varepsilon$

وبالمثل، البديل  $U$  لحاله (وهو أيضاً nullable) يعطي نفس النتيجة عبر  $A \rightarrow U \rightarrow \varepsilon$

$$V_2 = \{T, U, A\}$$

المجموعة الثالثة ( $V_3$ ): بفحص  $C$  و  $W$ :

$W \rightarrow T U C$  تحتاج إلى  $C$  لتتفرغ، و  $C$  ليست قابلة للتفرغ

$C \rightarrow U C$  تحتاج أيضاً إلى  $C$

نتوقف الخوارزمية هنا. المتحولات القابلة للتفرغ هي

$\{ A, T, U \}$

الخطوة 2: تعويض الأثر في كل قاعدة

قاعدة (1)  $T$ :

$$T \rightarrow b T \mid \varepsilon$$

بما أن  $T$  قابلة للتفرغ، نعوض  $\varepsilon$  مكان  $T$  في البديل  $b T$ ، فيبقى الرمز  $b$ . نحذف  $\varepsilon$  الصريحة فتصبح:

$$T \rightarrow b T \mid b$$

قاعدة (2)  $U$ :

$$U \rightarrow b U \mid a b W \mid \varepsilon$$

نعوض  $\varepsilon$  مكان  $U$  في البديل  $b U$ ، فيبقى الرمز  $b$ . نحذف  $\varepsilon$  الصريحة فتصبح:

$$U \rightarrow b U \mid b \mid a b W$$

قاعدة (3)  $W$ :

$$W \rightarrow T U C$$

البديل يحتوي على  $T$  و  $U$  وكلاهما Nullable نأخذ جميع التشكيلات الممكنة لاختفائهما:

• اختفاء  $T$  فقط: يتبقى  $U C$

• اختفاء  $U$  فقط: يتبقى  $T C$

• اختفاء كلاهما  $T$  و  $U$ : يتبقى  $C$

فتصبح:

$$W \rightarrow T U C \mid U C \mid T C \mid C$$

قاعدة (4)  $C$ :

$$C \rightarrow b T \mid U C$$

• في البديل  $b T$  عند اختفاء  $T$  يتبقى  $b$ .

• في البديل  $U C$  عند اختفاء  $U$  يتبقى  $C$ . يُهمل البديل  $C \rightarrow C$  عادةً لأنه لا يقدم جديداً.

فتصبح:

$$C \rightarrow b T \mid b \mid U C$$

قاعدة (5)  $A$ :

$$A \rightarrow A T \mid T \mid U$$

• في البديل  $A T$ :

◦ إذا اختفت  $T$ : يتبقى  $A$ .

◦ إذا اختفت  $A$ : يتبقى  $T$ .

• في البديل  $T$  لحاله:

حذفه بالكامل يعطي  $\epsilon$  لا نضيف بديلاً جديداً، لأن هذا الأثر يُعامل مثل قاعدة  $\epsilon$  الصريحة يلي رح تحذف بالنهاية أصلاً

• في البديل "U" لحاله:

نفس المنطق تماماً  $\leftarrow$  لا نضيف شي جديد

فتصبح القاعدة النهائية التي تجمع الأصلي مع التعويضات:

$$A \rightarrow AT \mid T \mid U$$

النتيجة النهائية للقواعد (بدون إبسيلون)

$$A \rightarrow AT \mid A \mid T \mid TU \mid U$$

$$T \rightarrow bT \mid b$$

$$U \rightarrow bU \mid b \mid abW$$

$$C \rightarrow bT \mid b \mid UC$$

$$W \rightarrow TUC \mid UC \mid TC \mid C$$

✍ ملاحظات هامة :

- أهمية الترتيب: الترتيب الإلزامي للاختصار أي قواعد Context-Free Grammar هو:

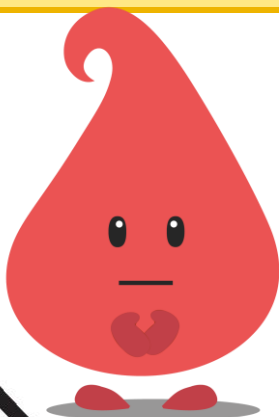
أولاً: حذف الرموز عديمة الفائدة (Useless Variables).

ثانياً: حذف الرموز غير القابلة للوصول (Inaccessible Variables).

ثالثاً: حذف قواعد الإبسيلون (Epsilon Productions)

لا يمكننا أبداً تغيير هذا الترتيب، كأن نبدأ بال Inaccessible قبل Useless

- الاستمرار بالتكرار: عند إضافة البدائل الناتجة عن التعويض، إذا ظهرت قاعدة تشير لنفسها مثل  $C \rightarrow C$ ، يمكنك تجاهلها فوراً لأنها لا تضيف أي تقدم وتعتبر تكراراً بلا طائل.



Ctrl + S my memory, please

## Turing Machine

تعتبر Turing Machine المستوى الرابع و الأعلى في تسلسل لغات وبرمجيات الحاسب متجاوزة Regular Languages، Context-Free Languages، و Pushdown Automata، ومروراً بالحسابات القابلة للحساب.

- هي النموذج الرياضي الأساسي الذي بُنيت عليه أجهزة الحاسب الحديثة التي نعرفها اليوم.
- تمتاز بقدرتها على حساب ومعالجة أي خوارزمية قابلة للحساب.

### الفرق الجوهرى بين آلة تورنغ والمستويات السابقة:

الفرق الأساسي يكمن في طريقة اتخاذ القرار (Decidability) (عند إعراب الكلمات أو معالجتها:

1. في المستويات السابقة: إذا كانت الكلمة **مقبولة** تعطينا الآلة قراراً فوراً، وإذا كانت **غير مقبولة** تعطينا قراراً بالرفض.

2. في آلة تورنغ:

إذا كانت الكلمة **مقبولة**، فهي حتماً ستعطينا قراراً بالقبول.

أما إذا كانت الكلمة **غير مقبولة**، فقد ترفضها فوراً، أو قد تدخل في حلقة لا نهائية وتستمر في العمل دون أن تعطي قراراً بالرفض (أي أن الآلة قد "تضيع" إذا لم تكن مصممة بحالة توقف صحيحة )

### المكونات الرياضية:

تتألف آلة تورنغ رياضياً من العناصر التالية:

- شريط ذاكرة (Tape): غير محدود السعة (يمكن القراءة والكتابة والتحرك يميناً ويساراً عليه بحرية خلافاً للمكدس المحدود).
- مجموعة الحالات (Q): تتضمن الحالة الابتدائية ( $q_0$ ) والحالات النهائية (F)
- مجموعة رموز الدخل ( $\Sigma$ ): الرموز التي تبدأ الآلة بقراءتها من الشريط.
- رموز الشريط ( $\Gamma$ ): الرموز المستخدمة أثناء المعالجة على الشريط (مثل رموز البدائل X, Y)
- رمز النهاية أو التوقف (B): يمثل الفراغ أو نهاية الرمز على الشريط (ويقاله Z0 في قاع المكدس)
- تابع الانتقال ( $\delta$ ): يحدد كيف تنتقل الآلة من حالة لأخرى، وماذا تكتب، واتجاه الحركة (يمين R أو يسار L)

تتعامل آلة تورنغ مع لغة تحتوي على عدد متساو من الأصفار يليه عدد مساو من الواحدات ( $0^n 1^n$ ) كالتالي:

### الفكرة الأساسية لخوارزمية آلة تورنغ للغة ( $0^n 1^n$ )

تعتمد الآلة على سياسة "الشطب المتبادل" المتدرج، وتعمل وفق التسلسل الدقيق التالي:

- شطب صفر واحد في كل دورة: تبدأ الآلة بقراءة صفر واحد فقط (0) من أقصى اليسار وتحوله إلى الرمز  $X$  (فهو لا تحول كل الأصفار دفعة واحدة). بعد ذلك، يتحرك رأس القراءة نحو اليمين متخطياً باقي الأصفار والرموز  $Y$  الموجودة في طريقه.
- البحث عن الواحد المقابل: تستمر الآلة بالتحرك يميناً حتى تصادف أول رقم 1، فتقوم فوراً بتحويله إلى  $Y$ .
- العودة نحو نقطة البداية: يعود رأس القراءة للتحرك يساراً، متخطياً الرموز  $Y$  والأصفار، حتى يصطدم بالرمز  $X$  الأخير الذي قام بكتابته.
- التكرار أو الانتقال للتحقق النهائي :

1. بمجرد الوقوف عند  $X$ ، تتحرك الآلة خطوة واحدة لليمين. إذا وجدت 0 جديداً، تعيد الدورة السابقة من البداية.

2. في حال انتهاء الأصفار: إذا تحركت الآلة لليمين ووجدت الرمز  $Y$  (بدلاً من 0) ، فهذا مؤشر صريح على أن جميع الأصفار قد تم شطبها بنجاح. هنا تنتقل الآلة إلى "مرحلة التحقق النهائي"، حيث تستمر بالتحرك يميناً متخطية كافة الرموز  $Y$  للتأكد من خلو الشريط من أي 1 إضافي، وعندما تصل إلى رمز الفراغ ( $B$ ) تُعلن قبول الكلمة.



## حالة الرفض أو الاستثناء (Exception) :

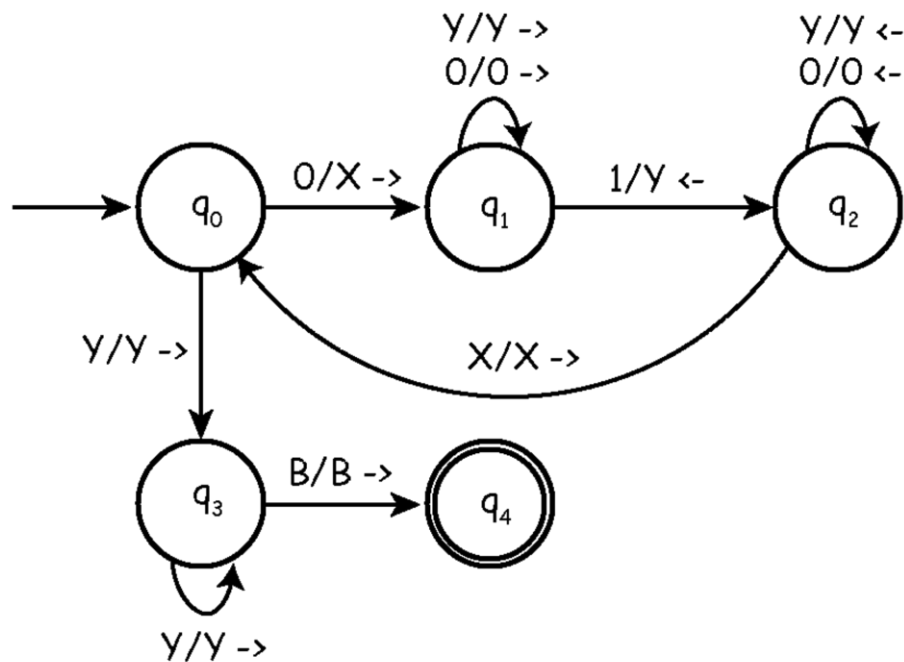
تعمل الآلة وفق قواعد وانتقالات حتمية وصارمة. إذا كانت الكلمة المدخلة خاطئة أو غير متوازنة (مثلاً أصفار أكثر من الواحدات، أو العكس، أو ترتيب خاطئ مثل 010) فإن الآلة ستصل إلى مرحلة تبحث فيها عن رمز معين ولن تجده.

**مثال:** أن تبحث الآلة عن 1 لتشطبه فتجد فراغاً (B)، أو أن تكون في مرحلة "التحقق النهائي" وتصادف رقماً إضافياً (1) بدلاً من الفراغ المطلوب.

في مثل هذه الحالات، لا تجد الآلة أي قاعدة انتقال (Transition) تناسب هذا الوضع العشوائي، فتتعثر الآلة فوراً وتتوقف عن العمل

تصدر (Exception)، مما يعني رفض الكلمة بشكل قاطع.

| State             | 0             | 1             | X             | Y             | B             |
|-------------------|---------------|---------------|---------------|---------------|---------------|
| $\rightarrow q_0$ | $(q_1, X, R)$ | —             | —             | $(q_3, Y, R)$ | —             |
| $q_1$             | $(q_1, 0, R)$ | $(q_2, Y, L)$ | —             | $(q_1, Y, R)$ | —             |
| $q_2$             | $(q_2, 0, L)$ | —             | $(q_0, X, R)$ | $(q_2, Y, L)$ | —             |
| $q_3$             | —             | —             | —             | $(q_3, Y, R)$ | $(q_4, B, R)$ |
| $* q_4$           | —             | —             | —             | —             | —             |



■ الحالة  $q_0$  : هي الحالة الابتدائية للبحث عن صفر من جهة اليسار فعندما تجده فإنها تبدله بـ  $X$  وتذهب إلى الحالة  $q_1$

■ الحالة  $q_1$  : هي الحالة التي تعبر فيها الآلة إلى اليمين عبر الأصفار والـ  $Ys'$  إذا رأت الآلة في هذه الحالة  $X$  أو  $B$  فإنها تموت عندما ترى 1 فإنها تبدله إلى  $Y$  وتدخل الحالة  $q_2$  وتبدأ التحرك يساراً

■ الحالة  $q_2$  : هي الحالة التي تعبر فيها الآلة إلى اليسار عبر الأصفار والـ  $Ys'$  عندما تصل الآلة إلى  $X$  فإنها تعود إلى الحالة  $q_0$  وتبدأ التحرك يمينا

■ إذا كانت الآلة في الحالة  $q_0$ :

1- وجدت صفر فإنها تكرر العملية

2- وجدت  $Y$  فإنها قد بدلت كل الأصفار بـ  $X$  وقد وجدت حتى اللحظة بأن كل 0 له مقابل 1 ويجب أن تقبل السلسلة ولذلك فإنها تدخل الحالة  $q_3$

■ الحالة  $q_3$  تبدأ فيها الآلة التحرك نحو اليمين عبر الـ  $Ys'$  حتى تصادف  $B$  فتدخل في الحالة  $q_4$  وتتقبل السلسلة. ولكن إن صادفت 1 في طريقها فهذا يعني بأن عدد الواحدات أكبر من عدد الأصفار فإن الآلة تموت

لنجرب الكلمات: 0011 و 00011

لنتفق أولاً على أن شريط الذاكرة يحتوي على الكلمة متبوعة برمز الفراغ  $B$  (والذي يمثل نهاية الكلمة)، وأن المؤشر يقف دائماً عند أول رمز من اليسار، والآلة تنطلق من الحالة الابتدائية  $q_0$ .

المثال الأول: الكلمة 0011 حالة نجاح وقبول

هذه السلسلة تتبع لغة  $(0^n 1^n)$  عدد الأصفار يساوي عدد الواحدات :  
الشكل المبدئي للشريط [ 0, 0, 1, 1, B ] :

الدورة الأولى (شطب الصفر الأول والواحد الأول):

1. الآلة في  $q_0$  تقرأ 0: تقوم بتبديله بالرمز  $X$ ، وتتحرك يمينا، وتنتقل للحالة  $q_1$ .

الشريط [ X, 0, 1, 1, B ]: المؤشر عند الصفر الثاني

2. الآلة في  $q_1$  تقرأ 0 :تخطاه (تتركه 0)، وتتحرك يمينا لتبحث عن أول واحد.  
الشريط [ X, 0, 1, 1, B ] :المؤشر عند الواحد الأول

3. الآلة في  $q_1$  تقرأ 1 :تقوم بتبديله بالرمز Y، وتتحرك يساراً، وتنتقل للحالة  $q_2$  لتبدأ رحلة العودة .  
الشريط [ X, 0, Y, 1, B ] :المؤشر عاد للصفر الثاني

4. الآلة في  $q_2$  تقرأ 0 :تخطاه يساراً.  
الشريط [ X, 0, Y, 1, B ] :المؤشر عند X

5. الآلة في  $q_2$  تقرأ X :بمجرد رؤيتها لـ X، تتحرك خطوة واحدة يمينا وتعود للحالة  $q_0$  لتبدأ دورة جديدة.  
الشريط [ X, 0, Y, 1, B ] :المؤشر عند الصفر الثاني

#### الدورة الثانية (شطب الصفر الثاني والواحد الثاني):

6. الآلة في  $q_0$  تقرأ 0 : تبدله بـ X، وتتحرك يمينا، وتنتقل للحالة  $q_1$ .  
الشريط [ X, X, Y, 1, B ] :المؤشر عند Y

7. الآلة في  $q_1$  تقرأ Y :تخطاها يمينا.  
الشريط [ X, X, Y, 1, B ] :المؤشر عند الواحد الثاني

8. الآلة في  $q_1$  تقرأ 1:تبدله بـ Y، وتتحرك يساراً، وتنتقل للحالة  $q_2$ .  
الشريط [ X, X, Y, Y, B ] :المؤشر عاد للـ Y الأولى

9. الآلة في  $q_2$  تقرأ Y :تخطاها يساراً.  
الشريط [ X, X, Y, Y, B ] :المؤشر عند الـ X الثانية

10. الآلة في  $q_2$  تقرأ X :تتحرك خطوة يمينا وتعود للحالة  $q_0$ .  
الشريط [ X, X, Y, Y, B ] :المؤشر عند الـ Y الأولى

#### مرحلة التحقق النهائي (التأكد من التوازن)

11. الآلة في  $q_0$  تقرأ Y :بما أنها لم تجد صفراً بل وجدت Y، فهذا يعني أن جميع الأصفار انتهت .تتركها Y، تتحرك يمينا وتنتقل للحالة  $q_3$  .  
الشريط [ X, X, Y, Y, B ] :المؤشر عند الـ Y الثانية



12. الآلة في  $q_3$  تقرأ  $Y$ : تتخطاها يميناً للتأكد من عدم وجود أي 1 إضافي.

الشريط [  $X, X, Y, Y, B$  ] : المؤشر وصل للفراغ  $B$

13. الآلة في  $q_3$  تقرأ  $B$ : تنتقل للحالة النهائية  $q_4$  وتتوقف .

النتيجة : الكلمة تنجح ومقبولة.



We're almost there

المثال الثاني: الكلمة 00011 حالة فشل ورفض

هنا نلاحظ أن عدد الأصفار (3) أكثر من عدد الواحدات (2). الشكل المبدئي للشريط [  $0, 0, 0, 1, 1, B$  ] :

الدورة الأولى:

- في  $q_0$  : نقرأ الصفر الأول، نحوله لـ  $X$ ، ونذهب يميناً للحالة  $q_1$ .
- في  $q_1$  : نعبّر الأصفار حتى نصل لأول 1، نحوله لـ  $Y$ ، ونعود يساراً للحالة  $q_2$ .
- في  $q_2$  : نعود يساراً حتى نصطدم بـ  $X$ ، نخطو يميناً ونعود لـ  $q_0$ .
- الشريط الآن [  $X, 0, 0, Y, 1, B$  ] : المؤشر يقف عند الصفر الثاني

الدورة الثانية:

- في  $q_0$  : نقرأ الصفر الثاني، نحوله لـ  $X$ ، ونذهب يميناً للحالة  $q_1$ .
- في  $q_1$  : نعبّر الـ 0 المتبقي والرمز  $Y$  حتى نصل للرقم 1 الثاني. نحوله لـ  $Y$ ، ونعود يساراً للحالة  $q_2$ .
- في  $q_2$  : نعود يساراً متخطين الـ  $Y$  والـ 0 حتى نصطدم بـ  $X$  الثانية، نخطو يميناً ونعود لـ  $q_0$ .
- الشريط الآن [  $X, X, 0, Y, Y, B$  ] : المؤشر يقف عند الصفر الثالث والأخير

الدورة الثالثة لحظة اكتشاف الخطأ وحدوث ال: (Exception)

■ الخطوة 1: الآلة في  $q_0$  تقرأ 0: تنفذ القاعدة فتحوله لـ  $X$ ، تتحرك يميناً وتنتقل للحالة  $q_1$ . الشريط الآن [ X, X, X, Y, Y, B ] :المؤشر يقف عند الـ  $Y$  الأولى

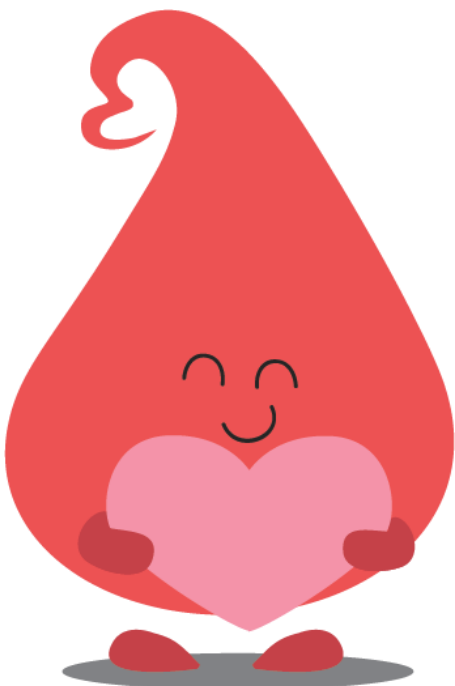
■ الخطوة 2: الآلة في  $q_1$  تقرأ  $Y$  :تخطاها وتستمر بالتحرك يميناً.

■ الخطوة 3: الآلة في  $q_1$  تقرأ  $Y$  الثانية :تخطاها وتتحرك يميناً الشريط الآن [ X, X, X, Y, Y, B ] :المؤشر يقف الآن عند رمز الفراغ  $B$ .

■ الخطوة 4 والأخيرة: الآلة في  $q_1$  تقرأ الرمز  $B$ :

هنا تبحث الآلة في القواعد المبرمجة لها: "ماذا أفعل إذا كنت في الحالة  $q_1$  وقرأت الرمز  $B$ ؟" المبرمج (القواعد المحددة) أعطى أوامر للحالة  $q_1$  بأن تقرأ 0، أو 1، أو  $Y$  فقط. لا توجد أي قاعدة أو مسار يخبر الآلة ماذا تفعل عند قراءة  $B$  في هذه الحالة! النتيجة: تصاب الآلة بالشلل التام (Exception)، وتتوقف عن العمل دون أن تصل للحالة النهائية  $q_4$ ، وبذلك تفشل الكلمة وترفض لأن عدد الواحدات أقل من عدد الأصفار.

THE END



إلى هنا نكون قد وصلنا إلى محطتنا الأخيرة  
في مقرر الأوتومات

لا تنسونا من صالح دعائكم ^^

فريق ال RBCs